

--- FILE: dsp_arc_detect.sv ---

```
1  `include "../include/apb_bus.sv"
2
3  module dsp_arc_detect #(
4      parameter DATA_WIDTH = 16,
5      parameter CNT_WIDTH = 16
6  )(
7      input logic          clk_i,
8      input logic          rst_ni,
9      input logic [DATA_WIDTH-1:0] adc_data_i,
10     input logic          adc_valid_i,
11     input logic          stream_restart_i,
12     APB_BUS              apb_slv,
13     output logic         irq_arc_o
14 );
15
16     // =====
17     // 1. REGISTER MAP
18     // =====
19     // 0x00 (RO): STATUS
20     //   [1:0] reg_status    00=SAFE, 01=WARN, 11=FIRE
21     //   [2]  irq_arc_o      live alarm output
22     //   [3]  fire_latched   sticky event flag
23     //   [4]  sample_pair_valid DSP has seen at least 2 samples
24     // 0x04 (RW): BASE_THRESH (compat with old DIFF_THRESH address)
25     // 0x08 (RW): INT_LIMIT
26     // 0x0C (RW): DECAY_RATE
27     // 0x10 (RW): BASE_ATTACK
28     // 0x14 (RO): CURRENT_DIFF_ABS
29     // 0x18 (RO): CURRENT_INTEGRATOR
30     // 0x1C (RO): PEAK_DIFF_ABS
31     // 0x20 (RO): PEAK_INTEGRATOR
32     // 0x24 (RO): EVENT_COUNT
33     // 0x28 (WO): CLEAR
34     //   [0] clear fire_latched
35     //   [1] clear diff / peak telemetry
36     //   [2] clear event_count
37     //   [3] reserved
38     //   [4] clear stream_restart_count
39     // 0x2C (RW): EXCESS_SHIFT
40     // 0x30 (RW): ATTACK_CLAMP
41     // 0x34 (RO): CURRENT_ATTACK_STEP
42     // 0x38 (RW): WIN_LEN
43     // 0x3C (RW): SPIKE_SUM_WARN
44     //   0 disables density gating for WARN
45     // 0x40 (RW): SPIKE_SUM_FIRE
46     //   0 disables density-based FIRE
47     // 0x44 (RO): CURRENT_SPIKE_SUM
48     // 0x48 (RO): PEAK_SPIKE_SUM
49     // 0x4C (RW): PEAK_DIFF_FIRE_THRESH
50     // 0x50 (RW): ALPHA_SHIFT
51     // 0x54 (RW): GAIN_SHIFT
52     // 0x58 (RO): CURRENT_NOISE_FLOOR
53     // 0x5C (RO): EFFECTIVE_THRESH
54     // 0x60 (RO): STREAM_STATUS
55     //   [0] stream_restart_i (live)
56     //   [1] detector_holdoff_active
57     //   [31:2] reserved
58     // 0x64 (RO): STREAM_RESTART_COUNT
59     // 0x68 (RW): HOT_BASE
60     // 0x6C (RW): HOT_ATTACK
61     // 0x70 (RW): HOT_DECAY
62     // 0x74 (RW): HOT_LIMIT
63     // 0x78 (RW): ENV_SHIFT
64     // 0x7C (RO): CURRENT_ENV_LP
65     // 0x80 (RO): CURRENT_HOTSPOT_SCORE
66     // 0x84 (RW): ZERO_BAND
67     // 0x88 (RW): QUIET_MIN
68     // 0x8C (RW): QUIET_MAX
69     // 0x90 (RO): CURRENT_QUIET_LEN
70     // 0x94 (RO): LAST_ZERO_GAP
71     // 0x98 (RO): LAST_FIRE_DIFF
72     // 0x9C (RO): LAST_FIRE_INT
73     // 0xA0 (RO): LAST_CAUSE_CODE
74     //   0 = none
75     //   1 = arc_by_density
76     //   2 = arc_by_standard
```

```

77 // 3 = thermal
78 // 4 = reserved (legacy data_fault slot)
79 // 5 = quiet_zone
80 // 0xA4 (RW): PROFILE_CTRL
81 // [3:0] active profile id (write low nibble to apply)
82 // [7:4] boot profile id
83
84
85
86
87 // có ch·a ··nh ngh·a ··a ch· thanh ghi APB
88 localparam logic [7:0] ADDR_STATUS = 8'h00;
89 localparam logic [7:0] ADDR_DIFF_THRESH = 8'h04;
90 localparam logic [7:0] ADDR_INT_LIMIT = 8'h08;
91 localparam logic [7:0] ADDR_DECAY_RATE = 8'h0C;
92 localparam logic [7:0] ADDR_BASE_ATTACK = 8'h10;
93 localparam logic [7:0] ADDR_CUR_DIFF = 8'h14;
94 localparam logic [7:0] ADDR_CUR_INT = 8'h18;
95 localparam logic [7:0] ADDR_PEAK_DIFF = 8'h1C;
96 localparam logic [7:0] ADDR_PEAK_INT = 8'h20;
97 localparam logic [7:0] ADDR_EVENT_COUNT = 8'h24;
98 localparam logic [7:0] ADDR_CLEAR = 8'h28;
99 localparam logic [7:0] ADDR_EXCESS_SHIFT = 8'h2C;
100 localparam logic [7:0] ADDR_ATTACK_CLAMP = 8'h30;
101 localparam logic [7:0] ADDR_CUR_ATTACK = 8'h34;
102 localparam logic [7:0] ADDR_WIN_LEN = 8'h38;
103 localparam logic [7:0] ADDR_SPIKE_WARN = 8'h3C;
104 localparam logic [7:0] ADDR_SPIKE_FIRE = 8'h40;
105 localparam logic [7:0] ADDR_CUR_SPIKE_SUM = 8'h44;
106 localparam logic [7:0] ADDR_PEAK_SPIKE_SUM = 8'h48;
107 localparam logic [7:0] ADDR_PEAK_DIFF_FIRE = 8'h4C;
108 localparam logic [7:0] ADDR_ALPHA_SHIFT = 8'h50;
109 localparam logic [7:0] ADDR_GAIN_SHIFT = 8'h54;
110 localparam logic [7:0] ADDR_CUR_NOISE = 8'h58;
111 localparam logic [7:0] ADDR_EFFECTIVE_THRESH = 8'h5C;
112 localparam logic [7:0] ADDR_STREAM_STATUS = 8'h60;
113 localparam logic [7:0] ADDR_STREAM_RESTART_COUNT = 8'h64;
114 localparam logic [7:0] ADDR_HOT_BASE = 8'h68;
115 localparam logic [7:0] ADDR_HOT_ATTACK = 8'h6C;
116 localparam logic [7:0] ADDR_HOT_DECAY = 8'h70;
117 localparam logic [7:0] ADDR_HOT_LIMIT = 8'h74;
118 localparam logic [7:0] ADDR_ENV_SHIFT = 8'h78;
119 localparam logic [7:0] ADDR_CUR_ENV_LP = 8'h7C;
120 localparam logic [7:0] ADDR_CUR_HOTSPOT = 8'h80;
121 localparam logic [7:0] ADDR_ZERO_BAND = 8'h84;
122 localparam logic [7:0] ADDR_QUIET_MIN = 8'h88;
123 localparam logic [7:0] ADDR_QUIET_MAX = 8'h8C;
124 localparam logic [7:0] ADDR_CUR_QUIET_LEN = 8'h90;
125 localparam logic [7:0] ADDR_LAST_ZERO_GAP = 8'h94;
126 localparam logic [7:0] ADDR_LAST_FIRE_DIFF = 8'h98;
127 localparam logic [7:0] ADDR_LAST_FIRE_INT = 8'h9C;
128 localparam logic [7:0] ADDR_LAST_CAUSE = 8'hA0;
129 localparam logic [7:0] ADDR_PROFILE_CTRL = 8'hA4;
130
131 localparam logic [3:0] CAUSE_NONE = 4'd0;//
132 localparam logic [3:0] CAUSE_ARC_DENSITY = 4'd1;//
133 localparam logic [3:0] CAUSE_ARC_STANDARD = 4'd2;//khi debug b·n không ch· bi·t ·có l·i·, mà còn bi·t l·i· ·n t·: arc th·ng arc theo m·t ·· spike
134 localparam logic [3:0] CAUSE_THERMAL = 4'd3;//
135 localparam logic [3:0] CAUSE_QUIET_ZONE = 4'd5;//
136 localparam logic [3:0] PROFILE_SAFE_RESET = 4'd0;
137 localparam logic [3:0] PROFILE_ARC_BALANCED = 4'd1;
138 localparam logic [3:0] PROFILE_THERMAL_BAL = 4'd2;
139 localparam logic [3:0] PROFILE_LAB_FULL = 4'd3;
140 localparam logic [3:0] DEFAULT_BOOT_PROFILE = PROFILE_ARC_BALANCED;
141
142 localparam [15:0] DEFAULT_BASE_THRESH = 16'd50;
143 localparam [15:0] DEFAULT_LIMIT = 16'd1000;
144 localparam [7:0] DEFAULT_DECAY = 8'd1;
145 localparam [15:0] DEFAULT_BASE_ATTACK = 16'd10;
146 localparam [4:0] DEFAULT_EXCESS_SHIFT = 5'd4;
147 localparam [15:0] DEFAULT_ATTACK_CLAMP = 16'd15;
148 localparam logic [6:0] DEFAULT_WIN_LEN = 7'd32;
149 localparam logic [6:0] DEFAULT_SPIKE_SUM_WARN = 7'd0;
150 localparam logic [6:0] DEFAULT_SPIKE_SUM_FIRE = 7'd0;
151 localparam logic [15:0] DEFAULT_PEAK_DIFF_FIRE_THRESH = 16'd75;
152 localparam logic [4:0] DEFAULT_ALPHA_SHIFT = 5'd2;
153 localparam logic [4:0] DEFAULT_GAIN_SHIFT = 5'd16;

```

```

154 localparam logic [15:0] DEFAULT_HOT_BASE = 16'd500;
155 localparam logic [15:0] DEFAULT_HOT_ATTACK = 16'd32;
156 localparam logic [15:0] DEFAULT_HOT_DECAY = 16'd4;
157 localparam logic [15:0] DEFAULT_HOT_LIMIT = 16'd96;
158 localparam logic [4:0] DEFAULT_ENV_SHIFT = 5'd4;
159 localparam logic [15:0] DEFAULT_ZERO_BAND = 16'd0;
160 localparam logic [7:0] DEFAULT_QUIET_MIN = 8'd2;
161 localparam logic [7:0] DEFAULT_QUIET_MAX = 8'd6;
162 localparam logic [6:0] MAX_WIN_LEN_CFG = 7'd64;
163
164 localparam int unsigned ACC_W = CNT_WIDTH + 1;
165 localparam int unsigned MAX_WIN_LEN = 64;
166 localparam int unsigned SPIKE_SUM_W = $clog2(MAX_WIN_LEN + 1);
167 localparam int unsigned QUIET_W = 8;
168 localparam int unsigned QUIET_CONF_W = 2;
169 localparam logic [QUIET_CONF_W-1:0] QUIET_CONF_FIRE_LEVEL = 2'd2;
170 localparam int unsigned RESTART_HOLDOFF_SAMPLES = 1;
171 localparam int unsigned RESTART_HOLDOFF_W = (RESTART_HOLDOFF_SAMPLES > 0) ? $clog2(RESTART_HOLDOFF_SAMPLES + 1) : 1;
172
173 logic [15:0] reg_base_thresh;
174 logic [15:0] reg_diff_threshold; // Backward-compatible alias for legacy TB/debug
175 logic [15:0] reg_int_limit;
176 logic [7:0] reg_decay_rate;
177 logic [15:0] reg_base_attack;
178 logic [4:0] reg_excess_shift;
179 logic [15:0] reg_attack_clamp;
180 logic [6:0] reg_win_len;
181 logic [SPIKE_SUM_W-1:0] reg_spike_sum_warn;
182 logic [SPIKE_SUM_W-1:0] reg_spike_sum_fire;
183 logic [15:0] reg_peak_diff_fire_thresh;
184 logic [4:0] reg_alpha_shift;
185 logic [4:0] reg_gain_shift;
186 logic [15:0] reg_hot_base;
187 logic [15:0] reg_hot_attack;
188 logic [15:0] reg_hot_decay;
189 logic [15:0] reg_hot_limit;
190 logic [4:0] reg_env_shift;
191 logic [15:0] reg_zero_band;
192 logic [QUIET_W-1:0] reg_quiet_min;
193 logic [QUIET_W-1:0] reg_quiet_max;
194 logic [1:0] reg_status;
195 logic [3:0] current_profile_q;
196
197 // =====
198 // 2. DSP CORE STATE / TELEMTRY
199 // =====
200 logic signed [DATA_WIDTH-1:0] sample_prev_q;
201 logic sample_pair_valid;
202
203 logic signed [DATA_WIDTH:0] diff_raw_comb;
204 logic [DATA_WIDTH-1:0] diff_abs_comb;
205 logic [DATA_WIDTH-1:0] diff_abs;
206 logic [DATA_WIDTH-1:0] diff_excess_comb;
207 logic is_spike_detected;
208
209 logic [CNT_WIDTH-1:0] integrator;
210 logic [CNT_WIDTH-1:0] peak_integrator;
211 logic [DATA_WIDTH-1:0] peak_diff;
212 logic fire_latched;
213 logic [15:0] event_count;
214 logic [15:0] stream_restart_count;
215 logic [CNT_WIDTH-1:0] attack_step_q;
216 logic [MAX_WIN_LEN-1:0] spike_hist_q;
217 logic [SPIKE_SUM_W-1:0] spike_sum_q;
218 logic [SPIKE_SUM_W-1:0] peak_spike_sum;
219 logic [RESTART_HOLDOFF_W-1:0] restart_holdoff_q;
220
221 logic [CNT_WIDTH-1:0] diff_excess_ext;
222 logic [CNT_WIDTH-1:0] excess_shifted_comb;
223 logic [CNT_WIDTH-1:0] excess_attack_term;
224 logic [ACC_W-1:0] attack_step_acc;
225 logic [CNT_WIDTH-1:0] attack_step_comb;
226 logic [ACC_W-1:0] attack_candidate;
227 logic [CNT_WIDTH-1:0] integrator_after_attack;
228 logic [CNT_WIDTH-1:0] integrator_after_decay;
229 logic [CNT_WIDTH-1:0] integrator_next_comb;
230 logic [CNT_WIDTH-1:0] warn_threshold;

```

```

231 logic outgoing_spike_bit;
232 logic [SPIKE_SUM_W-1:0] spike_sum_next_comb;
233 logic [DATA_WIDTH-1:0] peak_diff_next_comb;
234 logic warn_condition_comb;
235 logic fire_condition_comb;
236 logic detector_holdoff_active;
237 logic detector_sample_blocked;
238 logic [DATA_WIDTH-1:0] noise_floor_q;
239 logic [DATA_WIDTH-1:0] abs_sample_comb;
240 logic [DATA_WIDTH-1:0] env_lp_q;
241 logic signed [DATA_WIDTH:0] noise_floor_error_comb;
242 logic signed [DATA_WIDTH:0] noise_floor_adjust_comb;
243 logic signed [DATA_WIDTH:0] noise_floor_acc_comb;
244 logic [DATA_WIDTH-1:0] noise_floor_next_comb;
245 logic signed [DATA_WIDTH:0] env_error_comb;
246 logic signed [DATA_WIDTH:0] env_adjust_comb;
247 logic signed [DATA_WIDTH:0] env_acc_comb;
248 logic [DATA_WIDTH-1:0] env_lp_next_comb;
249 logic [DATA_WIDTH-1:0] noise_floor_gain_term;
250 logic [DATA_WIDTH:0] effective_thresh_acc;
251 logic [DATA_WIDTH-1:0] effective_thresh_comb;
252 logic thermal_hot_comb;
253 logic [ACC_W-1:0] hotspot_attack_candidate;
254 logic [CNT_WIDTH-1:0] hotspot_after_attack;
255 logic [CNT_WIDTH-1:0] hotspot_after_decay;
256 logic [CNT_WIDTH-1:0] hotspot_score_q;
257 logic [CNT_WIDTH-1:0] hotspot_score_next_comb;
258 logic thermal_fire_comb;
259 logic [QUIET_W-1:0] quiet_len_q;
260 logic [QUIET_W-1:0] last_zero_gap_q;
261 logic [QUIET_CONF_W-1:0] quiet_confidence_q;
262 logic [DATA_WIDTH-1:0] quiet_recent_peak_q;
263 logic [DATA_WIDTH-1:0] abs_prev_sample_comb;
264 logic near_zero_comb;
265 logic sign_change_comb;
266 logic [QUIET_W-1:0] quiet_len_next_comb;
267 logic [QUIET_W-1:0] quiet_gap_capture_comb;
268 logic [DATA_WIDTH-1:0] quiet_recent_peak_next_comb;
269 logic quiet_zone_match_comb;
270 logic [QUIET_CONF_W-1:0] quiet_confidence_next_comb;
271 logic quiet_fire_comb;
272 logic density_fire_comb;
273 logic standard_arc_fire_comb;
274 logic [3:0] trip_cause_code_comb;
275 logic [DATA_WIDTH-1:0] last_fire_diff_q;
276 logic [CNT_WIDTH-1:0] last_fire_int_q;
277 logic [3:0] last_cause_code_q;
278 logic stage_a_sample_valid_q;
279 logic stage_a_pair_valid_q;
280 logic [DATA_WIDTH-1:0] stage_a_diff_abs_q;
281 logic [DATA_WIDTH-1:0] stage_a_noise_floor_next_q;
282 logic [DATA_WIDTH-1:0] stage_a_effective_thresh_q;
283 logic [QUIET_W-1:0] stage_a_quiet_len_next_q;
284 logic [QUIET_W-1:0] stage_a_quiet_gap_capture_q;
285 logic [DATA_WIDTH-1:0] stage_a_quiet_recent_peak_next_q;
286 logic [DATA_WIDTH-1:0] stage_a_env_lp_next_q;
287 logic stage_a_sign_change_q;
288
289 // =====
290 // 3. APB COMMAND DECODE
291 // =====
292 logic apb_access;
293 logic apb_write;
294 logic [7:0] apb_addr;
295 logic clear_latched_cmd;
296 logic clear_peaks_cmd;
297 logic clear_events_cmd;
298 logic clear_restart_count_cmd;
299
300 assign apb_access = apb_slv.psel && apb_slv.penable;
301 assign apb_write = apb_access && apb_slv.pwrite;
302 assign apb_addr = apb_slv.paddr[7:0];
303
304 assign clear_latched_cmd = apb_write && (apb_addr == ADDR_CLEAR) && apb_slv.pwdata[0];
305 assign clear_peaks_cmd = apb_write && (apb_addr == ADDR_CLEAR) && apb_slv.pwdata[1];
306 assign clear_events_cmd = apb_write && (apb_addr == ADDR_CLEAR) && apb_slv.pwdata[2];
307 assign clear_restart_count_cmd = apb_write && (apb_addr == ADDR_CLEAR) && apb_slv.pwdata[4];

```

```

308 // =====
309 // 4. STAGE A - FRONT-END FEATURE EXTRACTION
310 // =====
311 function automatic logic [DATA_WIDTH-1:0] abs_diff(
312     input logic signed [DATA_WIDTH:0] value
313 );
314     logic signed [DATA_WIDTH:0] magnitude;
315     begin
316         magnitude = (value < 0) ? -value : value;
317         abs_diff = magnitude[DATA_WIDTH-1:0];
318     end
319 endfunction
320
321 function automatic logic [6:0] clamp_win_len(input logic [31:0] raw_value);
322     logic [6:0] raw_small;
323     begin
324         raw_small = raw_value[6:0];
325         if (raw_small == 7'd0)
326             clamp_win_len = 7'd1;
327         else if (raw_small > MAX_WIN_LEN_CFG)
328             clamp_win_len = MAX_WIN_LEN_CFG;
329         else
330             clamp_win_len = raw_small;
331     end
332 endfunction
333
334 function automatic logic [SPIKE_SUM_W-1:0] clamp_win_len_small(input logic [31:0] raw_value);
335     logic [6:0] clamped_len;
336     begin
337         clamped_len = clamp_win_len(raw_value);
338         clamp_win_len_small = clamped_len[SPIKE_SUM_W-1:0];
339     end
340 endfunction
341
342 function automatic logic [SPIKE_SUM_W-1:0] clamp_spike_level(
343     input logic [31:0] raw_value,
344     input logic [6:0] limit_value
345 );
346     logic [SPIKE_SUM_W-1:0] raw_small;
347     logic [SPIKE_SUM_W-1:0] limit_small;
348     begin
349         raw_small = raw_value[SPIKE_SUM_W-1:0];
350         limit_small = limit_value[SPIKE_SUM_W-1:0];
351
352         if (raw_small > limit_small)
353             clamp_spike_level = limit_small;
354         else
355             clamp_spike_level = raw_small;
356     end
357 endfunction
358
359 function automatic logic [3:0] sanitize_profile_id(input logic [3:0] raw_profile);
360     begin
361         case (raw_profile)
362             PROFILE_SAFE_RESET,
363             PROFILE_ARC_BALANCED,
364             PROFILE_THERMAL_BAL,
365             PROFILE_LAB_FULL: sanitize_profile_id = raw_profile;
366             default: sanitize_profile_id = PROFILE_SAFE_RESET;
367         endcase
368     end
369 endfunction
370
371 task automatic clear_detector_runtime_state(input logic clear_events_i);
372     begin
373         sample_prev_q    <= '0;
374         sample_pair_valid <= 1'b0;
375         diff_abs         <= '0;
376         peak_diff        <= '0;
377         integrator       <= '0;
378         peak_integrator  <= '0;
379         attack_step_q    <= '0;
380         spike_hist_q     <= '0;
381         spike_sum_q      <= '0;
382         peak_spike_sum   <= '0;
383         noise_floor_q    <= '0;

```

```

385     env_lp_q          <= '0;
386     hotspot_score_q   <= '0;
387     quiet_len_q       <= '0;
388     last_zero_gap_q   <= '0;
389     quiet_confidence_q <= '0;
390     quiet_recent_peak_q <= '0;
391     fire_latched      <= 1'b0;
392     last_fire_diff_q  <= '0;
393     last_fire_int_q   <= '0;
394     last_cause_code_q <= CAUSE_NONE;
395     stage_a_sample_valid_q <= 1'b0;
396     stage_a_pair_valid_q <= 1'b0;
397     stage_a_diff_abs_q <= '0;
398     stage_a_noise_floor_next_q <= '0;
399     stage_a_effective_thresh_q <= '0;
400     stage_a_quiet_len_next_q <= '0;
401     stage_a_quiet_gap_capture_q <= '0;
402     stage_a_quiet_recent_peak_next_q <= '0;
403     stage_a_env_lp_next_q <= '0;
404     stage_a_sign_change_q <= 1'b0;
405     reg_status        <= 2'b00;
406     irq_arc_o         <= 1'b0;
407     restart_holdoff_q <= '0;
408     if (clear_events_i)
409         event_count <= 16'd0;
410 end
411 endtask
412
413 task automatic_apply_profile(input logic [3:0] raw_profile_id, input logic clear_events_i);
414     logic [3:0] profile_id;
415     begin
416         profile_id = sanitize_profile_id(raw_profile_id);
417         current_profile_q <= profile_id;
418
419         unique case (profile_id)
420             PROFILE_ARC_BALANCED: begin
421                 reg_base_thresh <= 16'd80;
422                 reg_int_limit <= DEFAULT_LIMIT;
423                 reg_decay_rate <= DEFAULT_DECAY;
424                 reg_base_attack <= DEFAULT_BASE_ATTACK;
425                 reg_excess_shift <= DEFAULT_EXCESS_SHIFT;
426                 reg_attack_clamp <= DEFAULT_ATTACK_CLAMP;
427                 reg_win_len <= 7'd32;
428                 reg_spike_sum_warn <= SPIKE_SUM_W(3);
429                 reg_spike_sum_fire <= SPIKE_SUM_W(20);
430                 reg_peak_diff_fire_thresh <= 16'd220;
431                 reg_alpha_shift <= DEFAULT_ALPHA_SHIFT;
432                 reg_gain_shift <= 5'd3;
433                 reg_hot_base <= DEFAULT_HOT_BASE;
434                 reg_hot_attack <= DEFAULT_HOT_ATTACK;
435                 reg_hot_decay <= DEFAULT_HOT_DECAY;
436                 reg_hot_limit <= DEFAULT_HOT_LIMIT;
437                 reg_env_shift <= DEFAULT_ENV_SHIFT;
438                 reg_zero_band <= 16'd6;
439                 reg_quiet_min <= 8'd2;
440                 reg_quiet_max <= 8'd4;
441             end
442             PROFILE_THERMAL_BAL: begin
443                 reg_base_thresh <= 16'd60;
444                 reg_int_limit <= DEFAULT_LIMIT;
445                 reg_decay_rate <= DEFAULT_DECAY;
446                 reg_base_attack <= DEFAULT_BASE_ATTACK;
447                 reg_excess_shift <= DEFAULT_EXCESS_SHIFT;
448                 reg_attack_clamp <= DEFAULT_ATTACK_CLAMP;
449                 reg_win_len <= 7'd32;
450                 reg_spike_sum_warn <= SPIKE_SUM_W(2);
451                 reg_spike_sum_fire <= SPIKE_SUM_W(14);
452                 reg_peak_diff_fire_thresh <= 16'd180;
453                 reg_alpha_shift <= DEFAULT_ALPHA_SHIFT;
454                 reg_gain_shift <= 5'd5;
455                 reg_hot_base <= 16'd420;
456                 reg_hot_attack <= 16'd32;
457                 reg_hot_decay <= 16'd4;
458                 reg_hot_limit <= 16'd80;
459                 reg_env_shift <= DEFAULT_ENV_SHIFT;
460                 reg_zero_band <= 16'd6;
461                 reg_quiet_min <= 8'd2;

```

```

462     reg_quiet_max      <= 8'd4;
463 end
464 PROFILE_LAB_FULL: begin
465     reg_base_thresh    <= 16'd45;
466     reg_int_limit      <= DEFAULT_LIMIT;
467     reg_decay_rate     <= DEFAULT_DECAY;
468     reg_base_attack    <= DEFAULT_BASE_ATTACK;
469     reg_excess_shift   <= DEFAULT_EXCESS_SHIFT;
470     reg_attack_clamp   <= DEFAULT_ATTACK_CLAMP;
471     reg_win_len        <= 7'd32;
472     reg_spike_sum_warn <= SPIKE_SUM_W(2);
473     reg_spike_sum_fire <= SPIKE_SUM_W(6);
474     reg_peak_diff_fire_thresh <= 16'd120;
475     reg_alpha_shift    <= DEFAULT_ALPHA_SHIFT;
476     reg_gain_shift     <= 5'd4;
477     reg_hot_base       <= 16'd360;
478     reg_hot_attack     <= 16'd40;
479     reg_hot_decay      <= 16'd4;
480     reg_hot_limit      <= 16'd72;
481     reg_env_shift      <= DEFAULT_ENV_SHIFT;
482     reg_zero_band      <= 16'd10;
483     reg_quiet_min      <= 8'd2;
484     reg_quiet_max      <= 8'd5;
485 end
486 default: begin
487     reg_base_thresh    <= DEFAULT_BASE_THRESH;
488     reg_int_limit      <= DEFAULT_LIMIT;
489     reg_decay_rate     <= DEFAULT_DECAY;
490     reg_base_attack    <= DEFAULT_BASE_ATTACK;
491     reg_excess_shift   <= DEFAULT_EXCESS_SHIFT;
492     reg_attack_clamp   <= DEFAULT_ATTACK_CLAMP;
493     reg_win_len        <= DEFAULT_WIN_LEN;
494     reg_spike_sum_warn <= DEFAULT_SPIKE_SUM_WARN[SPIKE_SUM_W-1:0];
495     reg_spike_sum_fire <= DEFAULT_SPIKE_SUM_FIRE[SPIKE_SUM_W-1:0];
496     reg_peak_diff_fire_thresh <= DEFAULT_PEAK_DIFF_FIRE_THRESH;
497     reg_alpha_shift    <= DEFAULT_ALPHA_SHIFT;
498     reg_gain_shift     <= DEFAULT_GAIN_SHIFT;
499     reg_hot_base       <= DEFAULT_HOT_BASE;
500     reg_hot_attack     <= DEFAULT_HOT_ATTACK;
501     reg_hot_decay      <= DEFAULT_HOT_DECAY;
502     reg_hot_limit      <= DEFAULT_HOT_LIMIT;
503     reg_env_shift      <= DEFAULT_ENV_SHIFT;
504     reg_zero_band      <= DEFAULT_ZERO_BAND;
505     reg_quiet_min      <= DEFAULT_QUIET_MIN;
506     reg_quiet_max      <= DEFAULT_QUIET_MAX;
507 end
508 endcase
509
510     clear_detector_runtime_state(clear_events_i);
511 end
512 endtask
513
514 assign diff_raw_comb = $signed({adc_data_i[DATA_WIDTH-1], adc_data_i}) -
515     $signed({sample_prev_q[DATA_WIDTH-1], sample_prev_q});
516 assign diff_abs_comb = abs_diff(diff_raw_comb);
517 assign abs_sample_comb = abs_diff($signed({adc_data_i[DATA_WIDTH-1], adc_data_i}));
518 assign abs_prev_sample_comb = abs_diff($signed({sample_prev_q[DATA_WIDTH-1], sample_prev_q}));
519 assign near_zero_comb = (reg_zero_band != '0) && adc_valid_i && (abs_sample_comb < reg_zero_band);
520 assign sign_change_comb = sample_pair_valid && adc_valid_i &&
521     (adc_data_i[DATA_WIDTH-1] != sample_prev_q[DATA_WIDTH-1]);
522
523 assign noise_floor_error_comb = $signed({1'b0, diff_abs_comb}) - $signed({1'b0, noise_floor_q});
524 assign noise_floor_adjust_comb = noise_floor_error_comb >>> reg_alpha_shift;
525 assign noise_floor_acc_comb = $signed({1'b0, noise_floor_q}) + noise_floor_adjust_comb;
526 assign env_error_comb = $signed({1'b0, abs_sample_comb}) - $signed({1'b0, env_lp_q});
527 assign env_adjust_comb = env_error_comb >>> reg_env_shift;
528 assign env_acc_comb = $signed({1'b0, env_lp_q}) + env_adjust_comb;
529
530 always_comb begin
531     if (!sample_pair_valid || !adc_valid_i) begin
532         noise_floor_next_comb = noise_floor_q;
533     end else if (noise_floor_acc_comb < 0) begin
534         noise_floor_next_comb = '0;
535     end else if (noise_floor_acc_comb > $signed({1'b0, {DATA_WIDTH{1'b1}}})) begin
536         noise_floor_next_comb = {DATA_WIDTH{1'b1}};
537     end else begin
538         noise_floor_next_comb = noise_floor_acc_comb[DATA_WIDTH-1:0];

```

```

539     end
540 end
541
542 always_comb begin
543     if (!adc_valid_i) begin
544         env_lp_next_comb = env_lp_q;
545     end else if (env_acc_comb < 0) begin
546         env_lp_next_comb = '0';
547     end else if (env_acc_comb > $signed({1'b0, {DATA_WIDTH{1'b1}}})) begin
548         env_lp_next_comb = {DATA_WIDTH{1'b1}};
549     end else begin
550         env_lp_next_comb = env_acc_comb[DATA_WIDTH-1:0];
551     end
552 end
553
554 assign noise_floor_gain_term = (reg_gain_shift >= DATA_WIDTH) ? '0 : (noise_floor_q >> reg_gain_shift);
555 assign effective_thresh_acc = {1'b0, reg_base_thresh} + {1'b0, noise_floor_gain_term};
556 assign effective_thresh_comb = effective_thresh_acc[DATA_WIDTH] ? {DATA_WIDTH{1'b1}}
557                               : effective_thresh_acc[DATA_WIDTH-1:0];
558
559 always_comb begin
560     quiet_len_next_comb = quiet_len_q;
561     if (adc_valid_i) begin
562         if (near_zero_comb) begin
563             if (quiet_len_q == {QUIET_W{1'b1}})
564                 quiet_len_next_comb = quiet_len_q;
565             else
566                 quiet_len_next_comb = quiet_len_q + QUIET_W(1);
567         end else begin
568             quiet_len_next_comb = '0';
569         end
570     end
571 end
572
573 assign quiet_gap_capture_comb = near_zero_comb ? quiet_len_next_comb : quiet_len_q;
574 assign quiet_recent_peak_next_comb =
575     (sample_pair_valid && adc_valid_i && (diff_abs_comb > quiet_recent_peak_q))
576     ? diff_abs_comb
577     : quiet_recent_peak_q;
578
579 // =====
580 // 5. STAGE B - DECISION / SCORING
581 // =====
582 // Phase 1 fix remains intact, but now spike evaluation is fed by
583 // registered Stage A features for a shorter critical path.
584 assign is_spike_detected = stage_a_pair_valid_q && (stage_a_diff_abs_q > stage_a_effective_thresh_q);
585
586 // Weighted attack upgrade:
587 // excess = max(0, diff_abs - threshold)
588 // attack = base_attack + min(attack_clamp, excess >> excess_shift)
589 assign diff_excess_comb = (stage_a_diff_abs_q > stage_a_effective_thresh_q)
590     ? (stage_a_diff_abs_q - stage_a_effective_thresh_q)
591     : '0;
592 assign diff_excess_ext = diff_excess_comb;
593 assign excess_shifted_comb = (reg_excess_shift >= CNT_WIDTH) ? '0
594                             : (diff_excess_ext >> reg_excess_shift);
595 assign excess_attack_term = (excess_shifted_comb > reg_attack_clamp)
596     ? CNT_WIDTH'(reg_attack_clamp)
597     : excess_shifted_comb;
598 assign attack_step_acc = ACC_W'(reg_base_attack) + ACC_W'(excess_attack_term);
599 assign attack_step_comb = attack_step_acc[CNT_WIDTH-1:0];
600
601 assign attack_candidate = ACC_W'(integrator) + ACC_W'(attack_step_comb);
602 assign integrator_after_attack = (attack_candidate >= ACC_W'(reg_int_limit))
603     ? CNT_WIDTH'(reg_int_limit)
604     : attack_candidate[CNT_WIDTH-1:0];
605 assign integrator_after_decay = (integrator <= CNT_WIDTH'(reg_decay_rate))
606     ? '0
607     : (integrator - CNT_WIDTH'(reg_decay_rate));
608 assign integrator_next_comb = is_spike_detected ? integrator_after_attack
609                                     : integrator_after_decay;
610 assign warn_threshold = CNT_WIDTH'(reg_int_limit >> 1);
611 assign outgoing_spike_bit = spike_hist_q[reg_win_len - 1];
612 assign peak_diff_next_comb = (stage_a_pair_valid_q && (stage_a_diff_abs_q > peak_diff))
613     ? stage_a_diff_abs_q
614     : peak_diff;
615

```

```

616 always_comb begin
617     spike_sum_next_comb = spike_sum_q;
618     if (stage_a_sample_valid_q) begin
619         if (is_spike_detected && !outgoing_spike_bit)
620             spike_sum_next_comb = spike_sum_q + SPIKE_SUM_W(1);
621         else if (!is_spike_detected && outgoing_spike_bit)
622             spike_sum_next_comb = spike_sum_q - SPIKE_SUM_W(1);
623     end
624 end
625
626 assign quiet_zone_match_comb =
627     stage_a_sign_change_q &&
628     (reg_zero_band != '0') &&
629     (stage_a_quiet_gap_capture_q >= reg_quiet_min) &&
630     (stage_a_quiet_gap_capture_q <= reg_quiet_max) &&
631     (stage_a_quiet_recent_peak_next_q >= reg_peak_diff_fire_thresh) &&
632     (spike_sum_next_comb >= ((reg_spike_sum_warn == '0') ? SPIKE_SUM_W(1) : reg_spike_sum_warn));
633
634 always_comb begin
635     quiet_confidence_next_comb = quiet_confidence_q;
636     if (quiet_zone_match_comb && (quiet_confidence_q != {QUIET_CONF_W(1'b1)}))
637         quiet_confidence_next_comb = quiet_confidence_q + QUIET_CONF_W(1);
638 end
639
640 assign quiet_fire_comb = (quiet_confidence_next_comb >= QUIET_CONF_FIRE_LEVEL);
641 assign density_fire_comb = (reg_spike_sum_fire != '0') &&
642     (spike_sum_next_comb >= reg_spike_sum_fire) &&
643     (peak_diff_next_comb >= reg_peak_diff_fire_thresh);
644 assign standard_arc_fire_comb = (integrator_next_comb >= CNT_WIDTH'(reg_int_limit));
645
646 assign warn_condition_comb = ((integrator_next_comb > warn_threshold) &&
647     ((reg_spike_sum_warn == '0') || (spike_sum_next_comb >= reg_spike_sum_warn))) ||
648     quiet_zone_match_comb;
649 assign fire_condition_comb = standard_arc_fire_comb ||
650     density_fire_comb ||
651     quiet_fire_comb;
652 assign thermal_hot_comb = (stage_a_env_lp_next_q > reg_hot_base);
653 assign hotspot_attack_candidate = ACC_W'(hotspot_score_q) + ACC_W'(reg_hot_attack);
654 assign hotspot_after_attack = (hotspot_attack_candidate >= ACC_W'(reg_hot_limit))
655     ? CNT_WIDTH'(reg_hot_limit)
656     : hotspot_attack_candidate[CNT_WIDTH-1:0];
657 assign hotspot_after_decay = (hotspot_score_q <= CNT_WIDTH'(reg_hot_decay))
658     ? '0
659     : (hotspot_score_q - CNT_WIDTH'(reg_hot_decay));
660 assign hotspot_score_next_comb = thermal_hot_comb ? hotspot_after_attack
661     : hotspot_after_decay;
662 assign thermal_fire_comb = (hotspot_score_next_comb >= CNT_WIDTH'(reg_hot_limit));
663 assign trip_cause_code_comb = thermal_fire_comb ? CAUSE_THERMAL :
664     quiet_fire_comb ? CAUSE_QUIET_ZONE :
665     density_fire_comb ? CAUSE_ARC_DENSITY :
666     standard_arc_fire_comb ? CAUSE_ARC_STANDARD :
667     CAUSE_NONE;
668 assign reg_diff_threshold = reg_base_thresh;
669 assign detector_holdoff_active = (restart_holdoff_q != '0');
670 assign detector_sample_blocked = detector_holdoff_active;
671
672 // =====
673 // 6. APB + PIPELINED DSP SEQUENTIAL LOGIC
674 // =====
675 always_ff @(posedge clk_i or negedge rst_ni) begin
676     if (!rst_ni) begin
677         apply_profile(DEFAULT_BOOT_PROFILE, 1'b1);
678         stream_restart_count <= 16'd0;
679
680         apb_slv.pready <= 1'b0;
681         apb_slv.prdata <= 32'd0;
682         apb_slv.pslverr <= 1'b0;
683     end else begin
684         apb_slv.pready <= 1'b0;
685         apb_slv.prdata <= 32'd0;
686         apb_slv.pslverr <= 1'b0;
687
688         if (apb_access) begin
689             apb_slv.pready <= 1'b1;
690
691             if (apb_write) begin
692                 case (apb_addr)

```

```

693 ADDR_DIFF_THRESH: reg_base_thresh <= apb_slv.pwdata[15:0];
694 ADDR_INT_LIMIT: reg_int_limit <= apb_slv.pwdata[15:0];
695 ADDR_DECAY_RATE: reg_decay_rate <= apb_slv.pwdata[7:0];
696 ADDR_BASE_ATTACK: reg_base_attack <= apb_slv.pwdata[15:0];
697 ADDR_EXCESS_SHIFT: reg_excess_shift <= apb_slv.pwdata[4:0];
698 ADDR_ATTACK_CLAMP: reg_attack_clamp <= apb_slv.pwdata[15:0];
699 ADDR_WIN_LEN: begin
700     reg_win_len <= clamp_win_len(apb_slv.pwdata);
701     reg_spike_sum_warn <= (reg_spike_sum_warn > clamp_win_len(apb_slv.pwdata)) ? clamp_win_len_small(apb_slv.pwdata) : reg_spike_sum_warn;
702     reg_spike_sum_fire <= (reg_spike_sum_fire > clamp_win_len(apb_slv.pwdata)) ? clamp_win_len_small(apb_slv.pwdata) : reg_spike_sum_fire;
703     spike_hist_q <= '0';
704     spike_sum_q <= '0';
705     peak_spike_sum <= '0';
706 end
707 ADDR_SPIKE_WARN: reg_spike_sum_warn <= clamp_spike_level(apb_slv.pwdata, reg_win_len);
708 ADDR_SPIKE_FIRE: reg_spike_sum_fire <= clamp_spike_level(apb_slv.pwdata, reg_win_len);
709 ADDR_PEAK_DIFF_FIRE: reg_peak_diff_fire_thresh <= apb_slv.pwdata[15:0];
710 ADDR_ALPHA_SHIFT: reg_alpha_shift <= apb_slv.pwdata[4:0];
711 ADDR_GAIN_SHIFT: reg_gain_shift <= apb_slv.pwdata[4:0];
712 ADDR_HOT_BASE: reg_hot_base <= apb_slv.pwdata[15:0];
713 ADDR_HOT_ATTACK: reg_hot_attack <= apb_slv.pwdata[15:0];
714 ADDR_HOT_DECAY: reg_hot_decay <= apb_slv.pwdata[15:0];
715 ADDR_HOT_LIMIT: reg_hot_limit <= apb_slv.pwdata[15:0];
716 ADDR_ENV_SHIFT: reg_env_shift <= apb_slv.pwdata[4:0];
717 ADDR_ZERO_BAND: reg_zero_band <= apb_slv.pwdata[15:0];
718 ADDR_QUIET_MIN: reg_quiet_min <= apb_slv.pwdata[QUIET_W-1:0];
719 ADDR_QUIET_MAX: reg_quiet_max <= apb_slv.pwdata[QUIET_W-1:0];
720 ADDR_PROFILE_CTRL: apply_profile(apb_slv.pwdata[3:0], 1'b0);
721 default: ;
722 endcase
723 end else begin
724     case (apb_addr)
725         ADDR_STATUS: apb_slv.prdata <= {27'd0, sample_pair_valid, fire_latched, irq_arc_o, reg_status};
726         ADDR_DIFF_THRESH: apb_slv.prdata <= {16'd0, reg_base_thresh};
727         ADDR_INT_LIMIT: apb_slv.prdata <= {16'd0, reg_int_limit};
728         ADDR_DECAY_RATE: apb_slv.prdata <= {24'd0, reg_decay_rate};
729         ADDR_BASE_ATTACK: apb_slv.prdata <= {16'd0, reg_base_attack};
730         ADDR_CUR_DIFF: apb_slv.prdata <= {16'd0, diff_abs};
731         ADDR_CUR_INT: apb_slv.prdata <= {16'd0, integrator};
732         ADDR_PEAK_DIFF: apb_slv.prdata <= {16'd0, peak_diff};
733         ADDR_PEAK_INT: apb_slv.prdata <= {16'd0, peak_integrator};
734         ADDR_EVENT_COUNT: apb_slv.prdata <= {16'd0, event_count};
735         ADDR_EXCESS_SHIFT: apb_slv.prdata <= {27'd0, reg_excess_shift};
736         ADDR_ATTACK_CLAMP: apb_slv.prdata <= {16'd0, reg_attack_clamp};
737         ADDR_CUR_ATTACK: apb_slv.prdata <= {16'd0, attack_step_q};
738         ADDR_WIN_LEN: apb_slv.prdata <= {25'd0, reg_win_len};
739         ADDR_SPIKE_WARN: apb_slv.prdata <= {25'd0, reg_spike_sum_warn};
740         ADDR_SPIKE_FIRE: apb_slv.prdata <= {25'd0, reg_spike_sum_fire};
741         ADDR_CUR_SPIKE_SUM: apb_slv.prdata <= {25'd0, spike_sum_q};
742         ADDR_PEAK_SPIKE_SUM: apb_slv.prdata <= {25'd0, peak_spike_sum};
743         ADDR_PEAK_DIFF_FIRE: apb_slv.prdata <= {16'd0, reg_peak_diff_fire_thresh};
744         ADDR_ALPHA_SHIFT: apb_slv.prdata <= {27'd0, reg_alpha_shift};
745         ADDR_GAIN_SHIFT: apb_slv.prdata <= {27'd0, reg_gain_shift};
746         ADDR_CUR_NOISE: apb_slv.prdata <= {16'd0, noise_floor_q};
747         ADDR_EFFECTIVE_THRESH: apb_slv.prdata <= {16'd0, effective_thresh_comb};
748         ADDR_STREAM_STATUS: apb_slv.prdata <= {30'd0, detector_holdoff_active, stream_restart_i};
749         ADDR_STREAM_RESTART_COUNT: apb_slv.prdata <= {16'd0, stream_restart_count};
750         ADDR_HOT_BASE: apb_slv.prdata <= {16'd0, reg_hot_base};
751         ADDR_HOT_ATTACK: apb_slv.prdata <= {16'd0, reg_hot_attack};
752         ADDR_HOT_DECAY: apb_slv.prdata <= {16'd0, reg_hot_decay};
753         ADDR_HOT_LIMIT: apb_slv.prdata <= {16'd0, reg_hot_limit};
754         ADDR_ENV_SHIFT: apb_slv.prdata <= {27'd0, reg_env_shift};
755         ADDR_CUR_ENV_LP: apb_slv.prdata <= {16'd0, env_lp_q};
756         ADDR_CUR_HOTSPOT: apb_slv.prdata <= {16'd0, hotspot_score_q};
757         ADDR_ZERO_BAND: apb_slv.prdata <= {16'd0, reg_zero_band};
758         ADDR_QUIET_MIN: apb_slv.prdata <= {24'd0, reg_quiet_min};
759         ADDR_QUIET_MAX: apb_slv.prdata <= {24'd0, reg_quiet_max};
760         ADDR_CUR_QUIET_LEN: apb_slv.prdata <= {24'd0, quiet_len_q};
761         ADDR_LAST_ZERO_GAP: apb_slv.prdata <= {24'd0, last_zero_gap_q};
762         ADDR_LAST_FIRE_DIFF: apb_slv.prdata <= {16'd0, last_fire_diff_q};
763         ADDR_LAST_FIRE_INT: apb_slv.prdata <= {16'd0, last_fire_int_q};
764         ADDR_LAST_CAUSE: apb_slv.prdata <= {28'd0, last_cause_code_q};
765         ADDR_PROFILE_CTRL: apb_slv.prdata <= {24'd0, DEFAULT_BOOT_PROFILE, current_profile_q};
766         default: apb_slv.prdata <= 32'd0;
767     endcase
768 end
769 end

```

```

770
771 if (clear_latched_cmd) begin
772     fire_latched <= 1'b0;
773 end
774
775 if (clear_peaks_cmd) begin
776     diff_abs      <= '0;
777     peak_diff     <= '0;
778     peak_integrator <= '0;
779     attack_step_q <= '0;
780     peak_spike_sum <= '0;
781     quiet_len_q   <= '0;
782     last_zero_gap_q <= '0;
783     quiet_confidence_q <= '0;
784     quiet_recent_peak_q <= '0;
785 end
786
787 if (clear_events_cmd) begin
788     event_count <= 16'd0;
789     last_fire_diff_q <= '0;
790     last_fire_int_q <= '0;
791     last_cause_code_q <= CAUSE_NONE;
792 end
793
794 if (clear_restart_count_cmd) begin
795     stream_restart_count <= 16'd0;
796 end
797
798 if (stream_restart_i) begin
799     sample_prev_q      <= '0;
800     sample_pair_valid  <= 1'b0;
801     diff_abs           <= '0;
802     peak_diff          <= '0;
803     integrator         <= '0;
804     peak_integrator    <= '0;
805     attack_step_q      <= '0;
806     spike_hist_q       <= '0;
807     spike_sum_q        <= '0;
808     peak_spike_sum     <= '0;
809     noise_floor_q      <= '0;
810     env_lp_q           <= '0;
811     hotspot_score_q    <= '0;
812     quiet_len_q        <= '0;
813     last_zero_gap_q    <= '0;
814     quiet_confidence_q <= '0;
815     quiet_recent_peak_q <= '0;
816     last_fire_diff_q   <= '0;
817     last_fire_int_q    <= '0;
818     last_cause_code_q  <= CAUSE_NONE;
819     stage_a_sample_valid_q <= 1'b0;
820     stage_a_pair_valid_q <= 1'b0;
821     stage_a_diff_abs_q  <= '0;
822     stage_a_noise_floor_next_q <= '0;
823     stage_a_effective_thresh_q <= '0;
824     stage_a_quiet_len_next_q <= '0;
825     stage_a_quiet_gap_capture_q <= '0;
826     stage_a_quiet_recent_peak_next_q <= '0;
827     stage_a_env_lp_next_q <= '0;
828     stage_a_sign_change_q <= 1'b0;
829     reg_status         <= 2'b00;
830     irq_arc_o          <= 1'b0;
831     restart_holdoff_q  <= RESTART_HOLDOFF_SAMPLES[RESTART_HOLDOFF_W-1:0];
832     stream_restart_count <= stream_restart_count + 16'd1;
833 end else begin
834     if (adc_valid_i) begin
835         stage_a_sample_valid_q <= 1'b1;
836         stage_a_pair_valid_q  <= sample_pair_valid;
837         stage_a_diff_abs_q    <= diff_abs_comb;
838         stage_a_noise_floor_next_q <= noise_floor_next_comb;
839         stage_a_effective_thresh_q <= effective_thresh_comb;
840         stage_a_quiet_len_next_q <= quiet_len_next_comb;
841         stage_a_quiet_gap_capture_q <= quiet_gap_capture_comb;
842         stage_a_quiet_recent_peak_next_q <= quiet_recent_peak_next_comb;
843         stage_a_env_lp_next_q <= env_lp_next_comb;
844         stage_a_sign_change_q <= sign_change_comb;
845
846         sample_prev_q <= adc_data_i;

```

```

847     sample_pair_valid <= 1'b1;
848 end else begin
849     stage_a_sample_valid_q <= 1'b0;
850     stage_a_pair_valid_q <= 1'b0;
851     stage_a_sign_change_q <= 1'b0;
852 end
853
854 if (stage_a_sample_valid_q) begin
855     attack_step_q <= '0;
856     env_lp_q <= stage_a_env_lp_next_q;
857     hotspot_score_q <= hotspot_score_next_comb;
858     spike_hist_q <= {spike_hist_q[MAX_WIN_LEN-2:0], is_spike_detected};
859     spike_sum_q <= spike_sum_next_comb;
860
861     if (spike_sum_next_comb > peak_spike_sum)
862         peak_spike_sum <= spike_sum_next_comb;
863
864     quiet_len_q <= stage_a_quiet_len_next_q;
865     quiet_confidence_q <= quiet_confidence_next_comb;
866     if (stage_a_sign_change_q)
867         last_zero_gap_q <= stage_a_quiet_gap_capture_q;
868
869     if (stage_a_pair_valid_q) begin
870         if (stage_a_sign_change_q)
871             quiet_recent_peak_q <= '0;
872         else
873             quiet_recent_peak_q <= stage_a_quiet_recent_peak_next_q;
874     end
875
876     if (stage_a_pair_valid_q && !detector_sample_blocked) begin
877         diff_abs <= stage_a_diff_abs_q;
878         noise_floor_q <= stage_a_noise_floor_next_q;
879         if (stage_a_diff_abs_q > peak_diff)
880             peak_diff <= stage_a_diff_abs_q;
881
882         if (is_spike_detected)
883             attack_step_q <= attack_step_comb;
884
885         integrator <= integrator_next_comb;
886         if (integrator_next_comb > peak_integrator)
887             peak_integrator <= integrator_next_comb;
888
889         if (fire_condition_comb || thermal_fire_comb) begin
890             irq_arc_o <= 1'b1;
891             fire_latched <= 1'b1;
892             reg_status <= 2'b11;
893             last_fire_diff_q <= stage_a_diff_abs_q;
894             last_fire_int_q <= integrator_next_comb;
895             last_cause_code_q <= trip_cause_code_comb;
896
897             if (!irq_arc_o)
898                 event_count <= event_count + 16'd1;
899         end else if (irq_arc_o && ((integrator_next_comb != '0) || (hotspot_score_next_comb != '0))) begin
900             // Keep FIRE state while arc/thermal energy is still discharging.
901             reg_status <= 2'b11;
902         end else begin
903             if (warn_condition_comb)
904                 reg_status <= 2'b01;
905             else
906                 reg_status <= 2'b00;
907
908             if (integrator_next_comb == '0)
909                 irq_arc_o <= 1'b0;
910         end
911     end else if (stage_a_pair_valid_q && detector_sample_blocked) begin
912         if (detector_holdoff_active)
913             restart_holdoff_q <= restart_holdoff_q - 1'b1;
914     end
915 end
916 end
917 end
918 end
919
920 endmodule

```